

Interfaces + Dispatch

CS-3180

Style

- Today will be more focused on running code
- Do more code than slides
- Check the course GitHub for the code

Access

- In most OOP contexts
- Use the dot (.) operator
- Used in a few scenarios:

```
Student s1 = Student();  
// access GPA field  
s1.gpa;  
  
// call goToClass() method  
s1.goToClass();
```

Access

- In most OOP contexts
- Use the dot (.) operator
- Used in a few scenarios:
- Field Access

```
Student s1 = Student();  
// access GPA field  
s1.gpa;  
  
// call goToClass() method  
s1.goToClass();
```

Access

- In most OOP contexts
- Use the dot (.) operator
- Used in a few scenarios:
- Field Access
- Method Access

```
Student s1 = Student();  
// access GPA field  
s1.gpa;  
  
// call goToClass() method  
s1.goToClass();
```

Python Dot

- Python stores all its object data using `__dict__`
- Python dictionary containing all class information
- One for the instance: `p.__dict__`
- One for the Class: `Person.__dict__`

dict

- When you access something with dot
- You are actually indexing into this dictionary
- One of them at least
- Python looks up this value at runtime when you access them

```
instance.method() == instance.__dict__["method"]
```

```
class Person:
    def __init__(self, name, gpa):
        self.name = name
        self.gpa = gpa

    def go_to_class(self):
        print(self.name, " goes to class")

p = Person("Bob", 4.0)
p.go_to_class()
p.name
```

- This is how Python implements Dynamic Typing
- Under the hood at least

dict

- There is a lot of stuff in here
- Class.__dict__ contains a lot of function pointers
- instance.__dict__ contains just the data
- This is what self actually refers to

Using `__dict__`

- Since python uses this when we used dot
- We could be tricky and use it ourself
- For access

```
p: Person = Person("Bob", 3.9)
p.__dict__['new_value'] = "Hello"

print(p.__dict__)

print(f"New value was: {p.new_value}")
```

Using `__dict__`

- Since python uses this when we used dot
- We could be tricky and use it ourself
- For access
- Or for modification

```
p: Person = Person("Bob", 3.9)
p.__dict__['new_value'] = "Hello"

print(p.__dict__)

print(f"New value was: {p.new_value}")
```

Static Dispatch

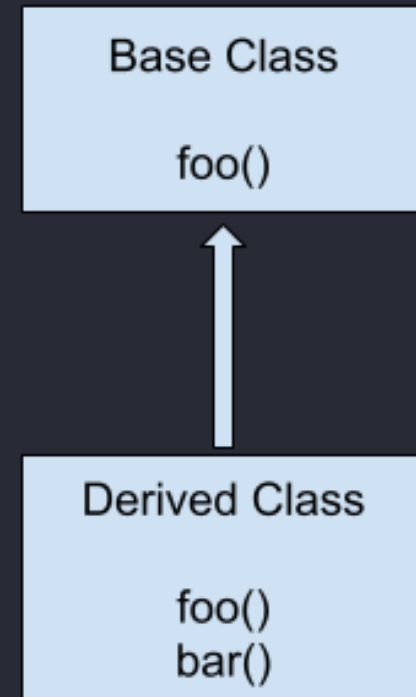
- If we were to translate this code over to Java
- It functions the same
- This is simple for data
- And for functions there is a *one-to-one* matching from:
 - Function name -> Implementation
- Static Dispatch: Address of callee can be resolved prior to execution

Java Complications

- Let's write some Java to see how this assumption breaks down
- Class Person
- Name, GPA, etc
- Let's override toString()
- See what happens when we don't

Inheritance

- Fundamental OOP concept
- A class can steal all of its behavior
- Java uses the "extends" keyword
- All objects implicitly extend from Object



What to call?

- Every Person IS AN Object
- If we had a method that accepted an object:
- Called toString()
- Which implementation gets called?

What to call?

- Every Person IS AN Object
- If we had a method that accepted an object:
- Called toString()
- Which implementation gets called?
- Person!

Typing

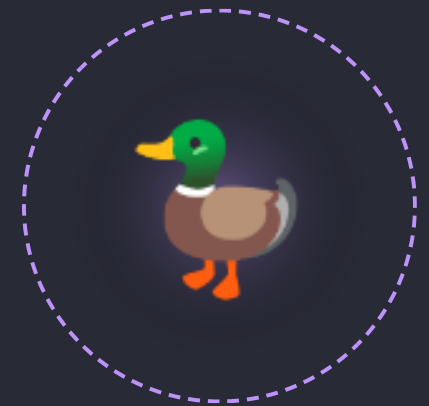
- Java separates data into its:
- Declared Type + Actual Type
- Static typing -> Each object know's what its actual type is
- Compiler only sees its declared type

```
// [DECLARED] [NAME] = [ACTUAL]
Object o = new Person();
o.[ONLY OBJECT ATTRIBUTES]

Person p = new Person()
p.[OBJECT + PERSON ATTRIBUTES]
```

Python Inheritance

- `class Derived(Base)` syntax
- Actually gives a much simpler solution
- Just change what the implementation is pointed to in `__dict__`
- Duck Typing will handle all of the edge cases*

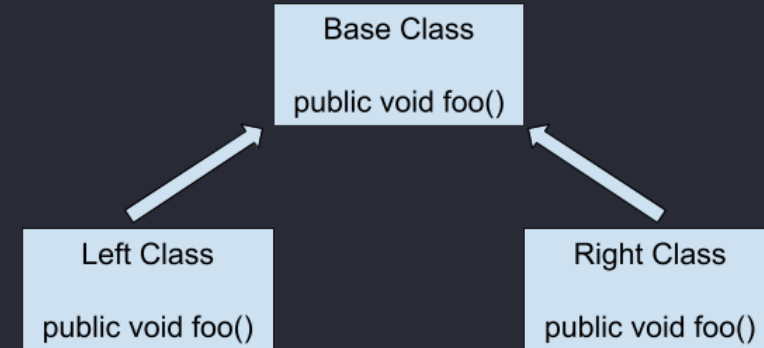


Diamond Problem

- Static Dispatch can still be achieved with Inheritance (at least in Java)
- Python and Java both prioritize execution of child-most methods
- This isn't always this so simple
- Let's present a specific scenario:

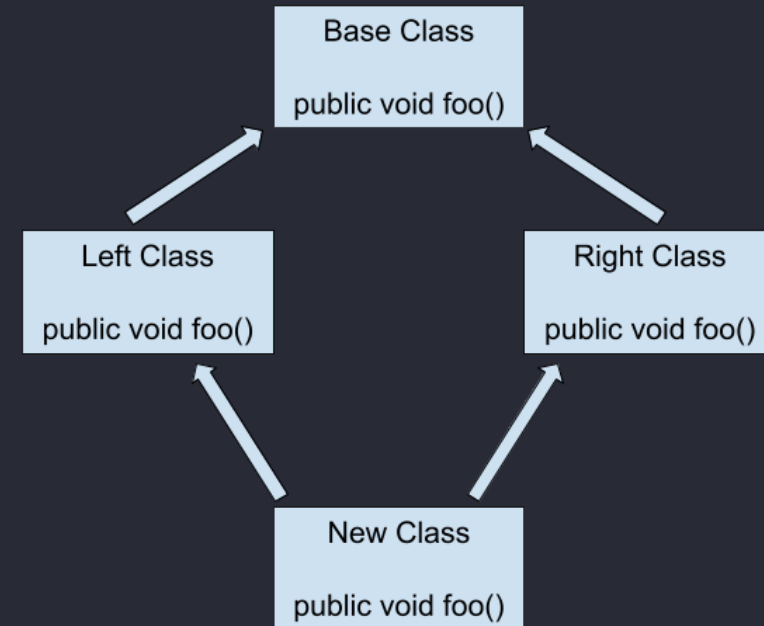
Multi-Inheritance

- Left and Right both override behavior
- Your code can figure out what the child-most implementation of foo() is



Multi-Inheritance

- If we added a class that derived behavior
- From both Left and Right (without override it)
- Which method should be called?



Diamond Problem

- Many solutions to this
- Java just prohibits multi-inheritance
- C++ looks this up based on class name?
- Python.... "Method Resolution Order" (MRO)

Method Resolution Order

The Python 2.3 Method Resolution Order

Note: This is a historical document, provided as an appendix to the official documentation. The Method Resolution Order discussed here was *introduced* in Python 2.3, but it is still used in later versions – including Python 3.

By [Michele Simionato](#).

Abstract: *This document is intended for Python programmers who want to understand the C3 Method Resolution Order used in Python 2.3. Although it is not intended for newbies, it is quite pedagogical with many worked out examples. I am not aware of other publicly available documents with the same scope, therefore it should be useful.*

Disclaimer:

I donate this document to the Python Software Foundation, under the Python 2.3 license. As usual in these circumstances, I warn the reader that what follows should be correct, but I don't give any warranty. Use it at your own risk and peril!

Acknowledgments:

All the people of the Python mailing list who sent me their support. Paul Foley who pointed out various imprecisions and made me to add the part on local precedence ordering. David Goodger for help with the formatting in reStructuredText. David Mertz for help with the editing. Finally, Guido van Rossum who enthusiastically added this document to the official Python 2.3 home-page.

The beginning

Felix qui potuit rerum cognoscere causas – Virgilius

Everything started with a post by Samuele Pedroni to the Python development mailing list [1]. In his post, Samuele showed that the Python 2.2 method resolution order is not monotonic and he proposed to replace it with the C3 method resolution order. Guido agreed with his arguments and therefore now Python 2.3 uses C3. The C3 method itself has nothing to do with Python, since it was invented by people working on Dylan and it is described in a paper intended for lispers [2]. The present paper gives a (hopefully) readable discussion of the C3 algorithm for Pythonistas who want to understand the reasons for the change.

First of all, let me point out that what I am going to say only applies to the *new style classes* introduced in Python 2.2: *classic classes* maintain their old method resolution order, depth first and then left to right. Therefore, there is no breaking of old code for classic classes; and even if in principle there could be breaking of code for Python 2.2 new style classes, in practice the cases in which the C3 resolution order differs from the Python 2.2 method resolution order are so rare that no real breaking of code is expected. Therefore:

Don't be scared!

Moreover, unless you make strong use of multiple inheritance and you have non-trivial hierarchies, you don't need to understand the C3 algorithm, and you can easily skip this paper. On the other hand, if you really want to know how multiple inheritance works, then this paper is for you. The good news is that things are not as complicated as you might expect.

Let me begin with some basic definitions.

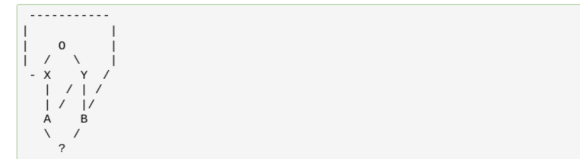
1. Given a class C in a complicated multiple inheritance hierarchy, it is a non-trivial task to specify the order in which methods are overridden, i.e. to specify the order of the ancestors of C.

1. Given a class C in a complicated multiple inheritance hierarchy, it is a non-trivial task to specify the order in which methods are overridden, i.e. to specify the order of the ancestors of C.
2. The list of the ancestors of a class C, including the class itself, ordered from the nearest ancestor to the furthest, is called the class precedence list or the *linearization* of C.
3. The *Method Resolution Order* (MRO) is the set of rules that construct the linearization. In the Python literature, the idiom “the MRO of C” is also used as a synonymous for the linearization of the class C.
4. For instance, in the case of single inheritance hierarchy, if C is a subclass of C1, and C1 is a subclass of C2, then the linearization of C is simply the list [C, C1, C2]. However, with multiple inheritance hierarchies, the construction of the linearization is more cumbersome, since it is more difficult to construct a linearization that respects *local precedence ordering* and *monotonicity*.
5. I will discuss the local precedence ordering later, but I can give the definition of monotonicity here. A MRO is monotonic when the following is true: *if C1 precedes C2 in the linearization of C, then C1 precedes C2 in the linearization of any subclass of C*. Otherwise, the innocuous operation of deriving a new class could change the resolution order of methods, potentially introducing very subtle bugs. Examples where this happens will be shown later.
6. Not all classes admit a linearization. There are cases, in complicated hierarchies, where it is not possible to derive a class such that its linearization respects all the desired properties.

Here I give an example of this situation. Consider the hierarchy

```
>>> 0 = object
>>> class X(0): pass
>>> class Y(0): pass
>>> class A(X,Y): pass
>>> class B(Y,X): pass
```

which can be represented with the following inheritance graph, where I have denoted with O the object class, which is the beginning of any hierarchy for new style classes:



In this case, it is not possible to derive a new class C from A and B, since X precedes Y in A, but Y precedes X in B, therefore the method resolution order would be ambiguous in C.

Python 2.3 raises an exception in this situation (TypeError: MRO conflict among bases Y, X) forbidding the naive programmer from creating ambiguous hierarchies. Python 2.2 instead does not raise an exception, but chooses an *ad hoc* ordering (CABXYO in this case).

The C3 Method Resolution Order

Let me introduce a few simple notations which will be useful for the following discussion. I will use the shortcut notation:

```
C1 C2 ... CN
```

to indicate the list of classes [C1, C2, ..., CN].

The *head* of the list is its first element:

```
head = C1
```

Method Resolution Order

The C3 Method Resolution Order

Let me introduce a few simple notations which will be useful for the following discussion. I will use the shortcut notation:

```
C1 C2 ... CN
```

to indicate the list of classes $[C1, C2, \dots, CN]$.

The *head* of the list is its first element:

```
head = C1
```

whereas the *tail* is the rest of the list:

```
tail = C2 ... CN.
```

I shall also use the notation:

```
C + (C1 C2 ... CN) = C C1 C2 ... CN
```

to denote the sum of the lists $[C] + [C1, C2, \dots, CN]$.

Now I can explain how the MRO works in Python 2.3.

Consider a class C in a multiple inheritance hierarchy, with C inheriting from the base classes $B1, B2, \dots, BN$. We want to compute the linearization $L[C]$ of the class C . The rule is the following:

the linearization of C is the sum of C plus the merge of the linearizations of the parents and the list of the parents.

In symbolic notation:

```
L[C(B1 ... BN)] = C + merge(L[B1] ... L[BN], B1 ... BN)
```

In particular, if C is the `object` class, which has no parents, the linearization is trivial:

```
L[object] = object.
```

However, in general one has to compute the merge according to the following prescription:

take the head of the first list, i.e. $L[B1][0]$; if this head is not in the tail of any of the other lists, then add it to the linearization of C and remove it from the lists in the merge, otherwise look at the head of the next list and take it, if it is a good head. Then repeat the operation until all the classes are removed or it is impossible to find good heads. In this case, it is impossible to construct the merge, Python 2.3 will refuse to create the class C and will raise an exception.

This prescription ensures that the merge operation *preserves* the ordering, if the ordering can be preserved. On the other hand, if the order cannot be preserved (as in the example of serious order disagreement discussed above) then the merge cannot be computed.

The computation of the merge is trivial if C has only one parent (single inheritance); in this case:

```
L[C(B)] = C + merge(L[B], B) = C + L[B]
```

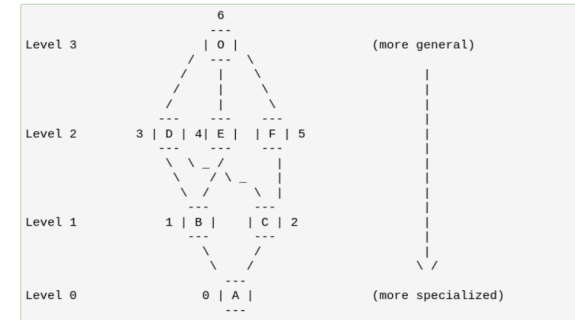
However, in the case of multiple inheritance things are more cumbersome and I don't expect you can understand the rule without a couple of examples :-)

Examples

First example. Consider the following hierarchy:

```
>>> 0 = object
>>> class F(0): pass
>>> class E(0): pass
>>> class D(0): pass
>>> class C(D,F): pass
>>> class B(D,E): pass
>>> class A(B,C): pass
```

In this case the inheritance graph can be drawn as:



The linearizations of `O`, `D`, `E` and `F` are trivial:

```
L[O] = 0
L[D] = D 0
L[E] = E 0
L[F] = F 0
```

The linearization of `B` can be computed as:

```
L[B] = B + merge(D0, E0, DE)
```

We see that `D` is a good head, therefore we take it and we are reduced to compute `merge(0, E0, DE)`. Now `O` is not a good head, since it is in the tail of the sequence `EO`. In this case the rule says that we have to skip to the next sequence. Then we see that `E` is a good head; we take it and we are reduced to compute `merge(0, 0)` which gives `O`. Therefore:

```
L[B] = B D E 0
```

Using the same procedure one finds:

```
L[C] = C + merge(D0, F0, DF)
      = C + D + merge(0, F0, F)
      = C + D + F + merge(0, 0)
      = C D F 0
```

Now we can compute:

```
L[A] = A + merge(BDE0, CDF0, BC)
      = A + B + merge(DE0, CDE0, C)
```

Method Resolution Order

Now we can compute:

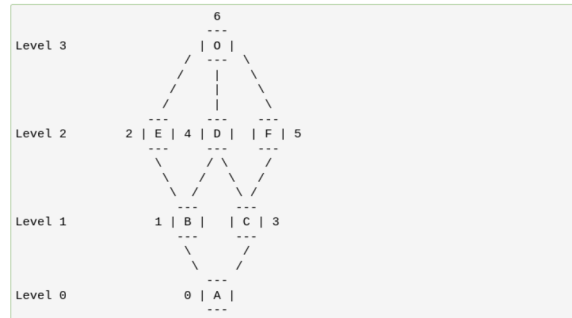
```
L[A] = A + merge(BDEO, CDFO, BC)
      = A + B + merge(DEO, CDFO, C)
      = A + B + C + merge(DEO, DFO)
      = A + B + C + D + merge(EO, FO)
      = A + B + C + D + E + merge(O, FO)
      = A + B + C + D + E + F + merge(O, O)
      = A B C D E F O
```

In this example, the linearization is ordered in a pretty nice way according to the inheritance level, in the sense that lower levels (i.e. more specialized classes) have higher precedence (see the inheritance graph). However, this is not the general case.

I leave as an exercise for the reader to compute the linearization for my second example:

```
>>> O = object
>>> class F(O): pass
>>> class E(O): pass
>>> class D(O): pass
>>> class C(D, F): pass
>>> class B(E, D): pass
>>> class A(B, C): pass
```

The only difference with the previous example is the change $B(D, E) \rightarrow B(E, D)$; however even such a little modification completely changes the ordering of the hierarchy:



Notice that the class E, which is in the second level of the hierarchy, precedes the class C, which is in the first level of the hierarchy, i.e. E is more specialized than C, even if it is in a higher level.

A lazy programmer can obtain the MRO directly from Python 2.2, since in this case it coincides with the Python 2.3 linearization. It is enough to invoke the `mro()` method of class A:

```
>>> A.mro()
[<class 'A'>, <class 'B'>, <class 'E'>,
 <class 'C'>, <class 'D'>, <class 'F'>,
 <class 'object'>]
```

Finally, let me consider the example discussed in the first section, involving a serious order disagreement. In this case, it is straightforward to compute the linearizations of O, X, Y, A and B:

```
L[O] = O
L[X] = X O
L[Y] = Y O
L[A] = A X Y O
L[B] = B Y X O
```

```
L[B, P21] = B C      # B precedes C : B's methods win
L[D, P21] = D A C B C # B follows C : C's methods win!
```

On the other hand, there are no problems with the Python 2.2 and 2.3 MROs, they give both:

```
L[D] = D A B C
```

Guido points out in his essay [3] that the classic MRO is not so bad in practice, since one can typically avoid diamonds for classic classes. But all new style classes inherit from `object`, therefore diamonds are unavoidable and inconsistencies shows up in every multiple inheritance graph.

The MRO of Python 2.2 makes breaking monotonicity difficult, but not impossible. The following example, originally provided by Samuele Pedroni, shows that the MRO of Python 2.2 is non-monotonic:

```
>>> class A(object): pass
>>> class B(object): pass
>>> class C(object): pass
>>> class D(object): pass
>>> class E(object): pass
>>> class K1(A, B, C): pass
>>> class K2(D, B, E): pass
>>> class K3(D, A): pass
>>> class Z(K1, K2, K3): pass
```

Here are the linearizations according to the C3 MRO (the reader should verify these linearizations as an exercise and draw the inheritance diagram ;-)

```
L[A] = A O
L[B] = B O
L[C] = C O
L[D] = D O
L[E] = E O
L[K1] = K1 A B C O
L[K2] = K2 D B E O
L[K3] = K3 D A O
L[Z] = Z K1 K2 K3 D A B C E O
```

Python 2.2 gives exactly the same linearizations for A, B, C, D, E, K1, K2 and K3, but a different linearization for Z:

```
L[Z, P22] = Z K1 K3 A K2 D B C E O
```

It is clear that this linearization is *wrong*, since A comes before D whereas in the linearization of K3 A comes *after* D. In other words, in K3 methods derived by D override methods derived by A, but in Z, which still is a subclass of K3, methods derived by A override methods derived by D! This is a violation of monotonicity. Moreover, the Python 2.2 linearization of Z is also inconsistent with local precedence ordering, since the local precedence list of the class Z is [K1, K2, K3] (K2 precedes K3), whereas in the linearization of Z K2 *follows* K3. These problems explain why the 2.2 rule has been dismissed in favor of the C3 rule.

The end

This section is for the impatient reader, who skipped all the previous sections and jumped immediately to the end. This section is for the lazy programmer too, who didn't want to exercise her/his brain. Finally, it is for the programmer with some hubris, otherwise s/he would not be reading a paper on the C3 method resolution order in multiple inheritance hierarchies :-). These three virtues taken all together (and *not* separately) deserve a prize: the prize is a short Python 2.2 script that allows you to compute the 2.3 MRO without risk to your brain. Simply change the last line to play with the various examples I have discussed in this paper.:

```
#<mro.py>

"""C3 algorithm by Samuele Pedroni (with readability enhanced by me)."""
```

Solutions

- (Please don't memorize that)
- Java brings in a new idea
- Interfaces
- Set of method declarations
- Without any implementations



Contract

- Validity of Interfaces are resolved at compile time
- If a class implements an interfaces
- Must implement its methods
- Compile-time contract

```
interface Drivable {  
    public void driveToWork();  
    public void driveToSchool();  
}
```

Declared Types

- Interfaces are only useful
- Because they can function as declared types
- Can pass them into methods

```
public static void commute(Drivable d){  
    d.driveToWork();  
    d.driveToSchool();  
}
```

Dynamic Dispatch

- Java's static typing will ensure this is a valid call
- At compile time
- But dynamically look up the correct implementation at runtime
- Just looking at this snippet -> No clue what implementation is called

```
public static void commute(Drivable d){  
    d.driveToWork();  
    d.driveToSchool();  
}
```

Dynamic Dispatch

- No longer a one-to-one mapping of declaration to implementation
- Dynamic Lookup of a method implementation
- Call this behavior dynamic dispatch
- Harder for a compiler to optimize
- Allows for SIGNIFICANT abstraction
- Basis for all OOP code at scale

Dispatch Implementations

- C++ uses the `virtual` keyword
- Look's up values in a VTABLE
- Python doesn't have interfaces
- Duck Typing = Implicit Interfaces?
- singledispatch + multipledispatch decorators?

Summary

- Inheritance makes function lookup complicated
- Static Dispatch: Resolve the callee prior to execution
- Dynamic Dispatch: Resolve the callee at runtime
- Some languages use a separate type for implementation contracts
- Key to resolving dynamic dispatch

Questions?